



项目-agent pay

项目概述

项目名称：AgentPay — AI Agent 间去中心化支付系统

一句话：基于 LangGraph 的多 Agent 系统 + Solidity 智能合约，实现 Agent 之间自动发现服务、链上托管支付、自动结算的演示系统。

亮点：

- 你的**多 Agent 架构经验**（LangGraph + MCP）+ **区块链支付**的结合+
- 纯 Vibe Coding（AI 辅助开发）完成，展示 AI 时代的开发范式+
- 技术栈覆盖全栈：智能合约 + Agent 编排 + 前端可视化+

应用场景

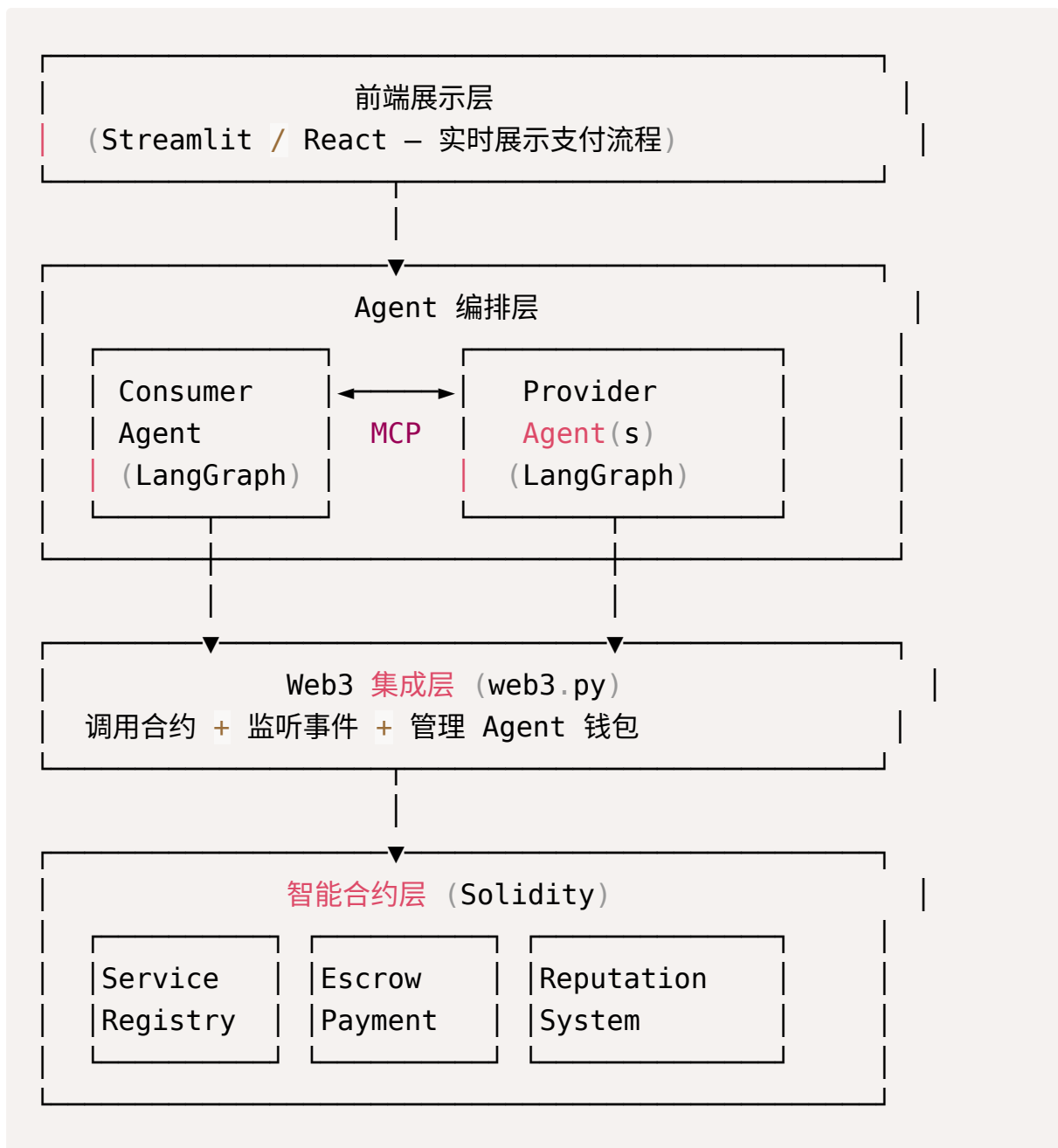
场景：个人股票分析 Agent 付费调用金融数据

你之前做的**股票投资顾问 Agent 系统**是手动配 API Key 的。这个项目让它变成自动化的：

1. 你的**基本面分析 Agent** 需要查 AAPL 最新财报+
2. 它在链上服务注册表中发现一个**金融数据 Agent** 提供"财报查询"服务标价 0.01 ETH/次+
3. 基本面分析 Agent 检查钱包余额足够 → 调用托管合约支付+
4. 合约锁住资金 → 通知金融数据 Agent 执行查询+
5. 金融数据 Agent 返回财报 JSON → 基本面分析 Agent 验证数据质量+
6. 验证通过 → 合约放款 / 验证失败或超时 → 自动退款+

整个流程不需要人介入，Agent 自己完成交易。

系统架构



技术栈

层级	技术	用途
Agent 编排	LangGraph (Python)	多 Agent 工作流定义、状态管理、MCP 工具调用
智能合约	Solidity • Hardhat	托管支付合约、服务注册合约、声誉合约
链上交互	web3.py	Agent 与合约交互、事件监听、交易签名

层级	技术	用途
本地网络	Hardhat Node / Anvil	本地测试链，部署合约
前端展示	Streamlit	实时展示 Agent 调用 + 支付流程的可视化看板
AI 辅助	Claude + Cursor	全流程 Vibe Coding，AI 生成代码 + 人工 review

项目文件结构

```

agent-pay/
├── contracts/                                # Solidity 智能合约
│   ├── ServiceRegistry.sol                 # 服务注册表
│   ├── EscrowPayment.sol                   # 托管支付合约
│   └── Reputation.sol                       # 声誉系统
├── agents/                                  # Agent 代码 (LangGraph)
│   ├── consumer_agent.py                   # 消费方 Agent (请求服务+付款)
│   ├── provider_agent.py                   # 供应方 Agent (提供数据+收款)
│   └── orchestrator.py                     # 编排层: Agent 发现 + 匹配 +
支付触发
├── web3_integration/                        # 链上交互
│   ├── wallet.py                           # Agent 钱包管理
│   ├── contract_interactions.py            # 合约调用封装
│   └── event_listener.py                   # 链上事件监听
├── frontend/                               # 前端展示
│   └── app.py                              # Streamlit 实时看板
├── tests/                                  # 测试
│   ├── test_contracts/                     # Hardhat 合约测试
│   └── test_agents/                         # Agent 逻辑测试
├── hardhat.config.js                       # Hardhat 配置
├── package.json                             # JS 依赖
└── README.md                               # 项目文档

```

核心合约设计

EscrowPayment.sol

```
// 核心逻辑：托管支付

// 1. Consumer 调用 requestService(providerId, fee)
//    → 转入 fee 金额，合约锁住
// 2. 合约触发 ServiceRequested 事件
// 3. Provider 监听到事件 → 执行服务 → 调用 deliverResult(dataHash)
// 4. Consumer 验证数据 → 调用 confirmDelivery()
//    → 合约放款给 Provider
// 5. 超时未确认 → Consumer 调用 refund() 退回资金
```

ServiceRegistry.sol

```
// 服务注册与发现
// 1. Provider 注册: registerService(name, description, price)
// 2. Consumer 查询: getService(name) → 返回 Provider 列表 + 价格
// 3. Provider 可更新/下架服务
```

简历可用描述

Project: AgentPay — Decentralized Payment System for AI Agents

Jun 2026 – Present / Personal Project

- Built a decentralized payment system enabling AI agents to autonomously discover, pay for, and verify services using LangGraph and Solidity smart contracts, demonstrating the intersection of multi-agent systems and blockchain payments.
- Designed a multi-agent architecture where a consumer agent discovers financial data providers through an on-chain service registry, initiates

escrow-based payments, and automatically settles after data delivery, with timeout and dispute handling.

- Developed three Solidity smart contracts (ServiceRegistry, EscrowPayment, Reputation) deployed on Hardhat local testnet, and a Streamlit dashboard that visualizes the complete agent-to-agent payment lifecycle.
- Entire project was built using AI-assisted development (Vibe Coding paradigm), showcasing proficiency in leveraging LLM tools for rapid prototyping and development.

面试话术

Q: 为什么想做这个项目?

A: 我之前做过多 Agent 股票分析系统，发现 Agent 之间调用服务都需要手动配 API Key、手动付费。如果未来 Agent 数量爆炸式增长，这种人工管理的方式不可持续。所以我想做一个"Agent 自己付钱给其他 Agent"的自动化方案，让 Agent 之间可以像人类一样进行交易。

Q: 技术难点在哪里?

A: 主要有两个难点。第一是 Agent 状态管理与链上交易的异步协调——Agent 发起支付后，区块确认需要时间，Agent 不能死等。我用了事件驱动的方式，Agent 监听链上事件来触发下一步。第二是托管合约的安全性设计，要防止重入攻击和超时逻辑的边界情况。

Q: 为什么叫 Vibe Coding 项目?

A: 整个项目的代码 80% 是由 AI 生成的 (Claude + Cursor)，我只负责架构设计、需求定义和代码 Review。这说明我懂得如何利用 AI 工具加速开发——在当前的技术环境下，这是一个很实用的能力。

后续规划

- ☐ Phase 1: Solidity 合约编写 + Hardhat 测试 (今天)
- ☐ Phase 2: LangGraph Agent 编排层 (明天上午)
- ☐ Phase 3: web3.py 集成层 (明天下午)
- ☐ Phase 4: Streamlit 前端看板

☐ Phase 5: 整体联调 + 写 README + 更新简历